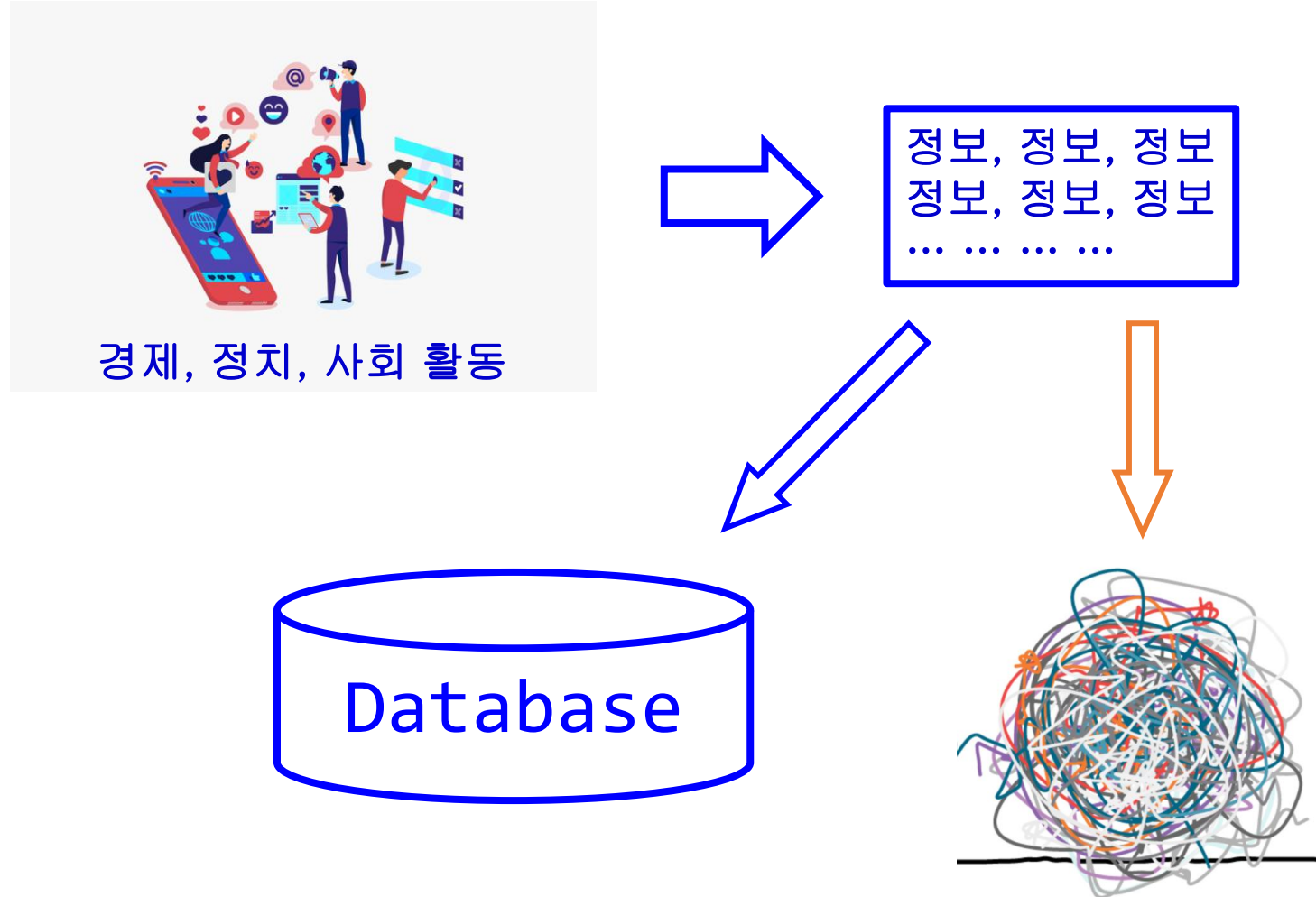


관계형 데이터베이스와 SQL

학습목표

1. 데이터베이스를 정의한다.
2. RDB와 RDBMS를 정의한다.
3. RDB의 구조를 이해한다.
4. 테이블의 구조를 학습한다.
5. 관계를 학습한다.
6. SQL를 개념을 학습한다.

Database(데이터베이스)



Database(데이터베이스)

- DBMS : Database Management System
 - Database를 구현하고 사용 가능하도록 작업 환경을 제공하는 소프트웨어
 - 계층형데이터베이스, 망형데이터베이스, 관계형데이터베이스
- RDBMS : Relational Database Management System
 - RDB(관계형 데이터베이스)를 관리하는 소프트웨어
 - Oracle, SQL Server, MySQL, DB2
 - 관계형데이터베이스 ≡ 정보(사회활동 부산물)

RDB(Relational Database)

- 테이블로 구성
 - 열과 행으로 구성(column, row)

이름	성별	생년월일	부서
유비	남	01610716	개발부
조조	남	01550101	총무부
관우	남	01620816	기획부
장비	남	01630916	영업부

→ 행(row)

↓ 열(컬럼 : column)

RDB(Relational Database)

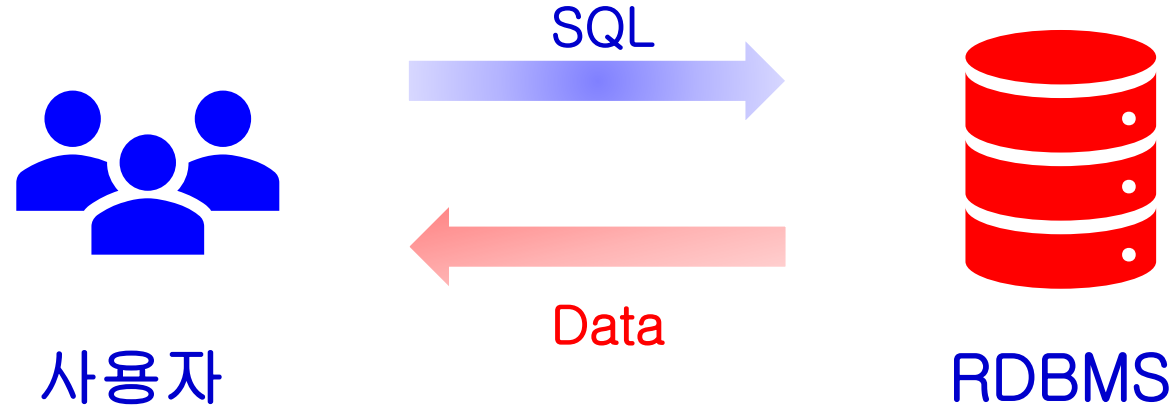
- 관계

이름	성별	생년월일	부서
유비	남	01610716	개발부
조조	남	01550101	총무부
관우	남	01620816	기획부
장비	남	01630916	영업부

조조가 근무하는 사무실은
몇 호인가요??

부서	인원수	사무실
개발부	20	101호
총무부	2	106호
기획부	4	109호
영업부	10	201호

SQL(Structured Query Language)



SQL : 정확한 데이터 출력(조작)

SQL(Structured Query Language)

- DML(Data Manipulation Language)
 - SELECT
 - INSERT, UPDATE, DELETE
- DDL(Data Definition Language)
 - CREATE, ALTER, DROP
 - RENAME
- DCL(Data Control Language)
 - GRANT, REVOKE
- TCL(Transaction Control Language)
 - COMMIT, ROLLBACK

SQL(Structured Query Language)

- 영어와 매우 유사한 언어
 - 단어의 구성과 순서가 고정
 - 자연어에 가까움(쉬움)
- 비 절차적 언어
 - 조건문(?), 순환문 없음.
 - 프로그래밍이 어려움
- 표준안이 제공된다.

실습 테이블

테이블	내용	테이블	내용
emp dept salgrade	사원 부서 급여등급	student professor course score scgrade	학생 교수 과목 기말고사점수 점수등급

테이블 상세 구조

emp(사원)		dept(부서)		salgrade(급여등급)		scgrade(점수등급)	
컬럼	내용	컬럼	내용	컬럼	내용	컬럼	내용
eno	사번	dno	부서번호	grade	등급	grade	등급
ename	이름	dname	부서명	losal	하한값	loscore	하한값
sex	성별	loc	지역	hisal	상한값	hiscore	상한값
job	업무						
mgr	사수사번						
hdate	입사일						
sal	급여						
comm	보너스						
dno	부서번호						

student(학생)		professor(교수)		course(과목)		score(기말고사점수)	
컬럼	내용	컬럼	내용	컬럼	내용	컬럼	내용
sno	학번	pno	교수번호	cno	과목번호	sno	학번
sname	이름	pname	교수이름	cname	과목명	cno	과목번호
sex	성별	section	소속학과	st_num	학점수	result	점수
syear	학년	orders	직위	pno	교수번호		
major	학과	hiredate	부임일				
avr	평점(4.0)						

SELECT문의 이해

학습목표

1. SELECT 문의 기능을 학습한다.
2. 기본 SELECT 문의 구조 학습한다.
3. 스키마의 테이블 목록을 검색한다.
4. 테이블 구조를 검색한다.
5. 원하는 데이터가 포함된 컬럼을 검색한다.

SELECT문의 기능

프로젝션(Projection)

선택(Selection)

조인(Join)

SELECT문의 구조

```
SELECT * | [DISTINCT] 컬럼|수식 [AS 별명], .....  
FROM 테이블명;
```

- SELECT 절에는 출력 대상을 기술한다.
 - * 단독으로만 쓰인다.
 - 컬럼, 수식
- FROM 절에는 테이블 명을 기술한다.
 - SELECT절과 FROM절은 생략할 수 없다.

특별한 SQL문

- 모든 테이블을 검색한다.

```
SELECT * FROM tab;
```

- 현재 유저가 소유한 테이블 목록을 확인한다.
- 유저, 스키마는 동일한 의미이다.

- 테이블 구조를 검색한다.

```
DESC 테이블;
```

- 테이블이 어떤 컬럼으로 구성되어 있는지 확인한다.

SQL 문의 구조

- SQL 문은 다양한 유연성을 가진 비절차적 언어이다. C언어나 자바 등과는 달리 다양한 순환 구조와 조건 구문을 갖지는 않지만 자연어에 꽤 가까운 언어임으로 입문시에 배우고 구사하는 것이 쉽다.
 - SQL 문은 반드시 ';'으로 끝난다.
 - SQL 문은 대소문자를 구별하지 않는다.
 - SQL 문은 여러 줄에 기술 가능하다.
 - . 들여 쓰기 가능하다.
 - SQL 문은 반드시 결과(출력)가 존재한다.

테이블 구조와 내용 검색

- 사원(emp) 테이블의 구조와 내용을 검색한다.

```
SQL> DESC emp;  
SQL> SELECT * FROM emp;
```

- 사원의 사번과 이름을 검색한다.

```
SQL> SELECT eno, ename  
2 FROM emp;
```

- 지문에 맞는 SELECT문을 검색한다.
- 지문을 해석하기 위해서는 테이블의 구조를 반드시 기억해야 한다.

올바른 데이터 검색

- 다음 SELECT 문을 비교해 보자.

```
SQL> SELECT ename, job FROM emp;
```

```
SQL> SELECT eno, ename, job FROM emp;
```

- 문법이 올바른 SQL문이라도 결과가 반드시 바른 것은 아니다.
- SQL문 뿐 아니라 검색 결과도 문제가 없어야 한다.
 - 검색 결과는 질의의 의미가 충족되어야 한다.
 - 검색 결과는 구분이 되어야 한다. – 무결성

다양한 검색

- 다음 검색 결과를 비교해 보자.

```
SQL> SELECT eno, ename, sal, comm  
2 FROM emp;
```

- 컬럼의 데이터 타입에 따라 맞춤위치가 달라진다.
- 문자형은 왼쪽 맞춤으로 숫자는 오른쪽 맞춤으로 검색된다.

테이블은 2차원 배열과 다르다.

부서 : 테이블		
부서번호	부서명	지역
01	총무	서울
02	기획	서울
10	ERP	대전
20	ISP	부산
30	ITEA	광주

SELECT 부서명 **FROM** 부서;

```
for i : 0 .. 4  
  print 부서[i][부서명]
```

부서[] : 배열			
	부서번호	부서명	지역
0	01	총무	서울
1	02	기획	서울
2	10	ERP	대전
3	20	ISP	부산
4	30	ITEA	광주

실습

1. student, course, prfessor score 테이블의 구조를 검색한다.
2. 모든 학생의 정보를 검색한다.
3. 모든 교수의 정보를 검색한다.
4. 모든 과목의 정보를 검색한다.
5. 모든 기말고사 점수를 검색한다
6. 학생들의 학과와 학년을 검색한다.
7. 각 과목의 이름을 검색한다.
8. 교수들의 직위를 검색한다.
9. 과목의 학점수를 검색한다.
10. 학생의 성별을 검색한다.

다양한 SELECT 문 활용

학습목표

1. 컬럼을 이용한 수식을 검색한다.
2. 별명을 이용해서 검색한다.
3. 별명의 다양한 규칙을 이해한다.
4. NULL의 특성에 대해서 이해한다.
5. NULL이 포함된 경우 사용되는 NVL() 함수를 이용한다.

수식 검색

- 수식을 검색한다.

```
SELECT [수식] FROM dual;
```

- SELECT 절에 수식을 사용 할 수 있다.
- SELECT 절에 컬럼이 없는 경우 FROM절에는 더미 테이블명인 dual을 이용한다.
 - SELECT문에 FROM절을 생략 할 수 없다.
- 다음 수식을 검색한다.
 - ex)
SELECT 2+3 FROM dual;
SELECT (2+3)*6 FROM dual;

별명을 이용한 검색

```
SELECT 컬럼명 AS "별명", .....  
FROM 테이블명;
```

- 별명은 헤더의 출력을 지정한다.
- AS는 생략 가능하다.
- "(이중인용부호)가 필요한 경우
 - . 공백이 포함된 경우
 - . 특수문자가 포함된 경우
 - . 대소문자를 구분하고자 할 경우

별명을 이용한 검색

```
SELECT eno AS "사원 번호", ename AS "이름"  
FROM emp;
```

```
SELECT dno 부서_번호, dname 부서명  
FROM dept;
```

```
SELECT eno 사번, sal*1.1 "10%인상급여"  
FROM emp;
```

수식을 이용한 검색

```
SELECT eno 사번, ename 이름, sal*12 연간_급여  
FROM emp;
```

```
SELECT eno 사번, ename 이름, sal*1.1 인상_급여  
FROM emp;
```

- 수식을 검색하는 경우 헤더에 수식이 출력됨으로 이런 경우 가급적 별명을 출력하면 가독성이 높아진다.
- 헤더는 응용 프로그램 개발시에 변수명이나 배열의 키로 이용됨으로 프로그램 개발시에는 이를 고려한다.

널이 포함된 연산

```
SELECT eno, ename, sal, comm, sal*12+comm  
FROM emp;
```

- NULL은 결정되지 않은 (알수 없는, 미지의,...) 값을 의미한다.
- NULL의 연산 결과는 NULL이다.
- NULL은 공백이나 0 또는 False와 다르다.
- 보안상 부정으로 간주될 수 있다.

• NVL 함수

```
...NVL(컬럼, 치환값) ...
```

- 컬럼의 값이 NULL이면 치환값으로 바꾼다.
- $sal*12+comm$
→ $sal*12+nvl(comm,0)$

실습

1. 각 학생의 평균 평점을 검색한다.(별명을 이용)
2. 각 과목의 학점수는 검색한다.(별명을 이용)
3. 각 교수의 지위를 검색한다.(별명을 이용)
4. 급여를 10% 인상했을 때 연간 지급되는 급여를 검색한다.
5. 현재 학생의 평균 평점은 4.0 만점이다. 이를 4.5 만점으로 환산해서 검색한다.
6. 급여가 10% 인상될 경우 각 사원의 연봉을 검색한다.
7. 1년 동안 지급되는 급여와 10% 인상되어 1년 동안 지급되는 급여 간에 차액을 검색한다.
8. 각 학생들의 평균평점을 4.5 만점으로 환산한 경우 각각 평점의 상승폭은 얼마인지 검색한다.

출력 화면 제어

검색 화면

SQL> select * from emp;

ENO	ENAME	SEX	JOB	MGR	HDATE
0001	안영희	여	경영지원		1991/01/01
0201	안영희	여	경영지원	0001	1991/02/01
0202	안영희	여	경영지원	0001	1991/12/01
0301	이소희	여	회계	0001	1992/02/01
0302	이소희	여	회계	0301	1991/03/02
1001	이소희	여	회계	0201	1991/02/01
1002	이소희	여	회계	0201	1992/03/03
1003	이소희	여	회계	0201	1995/02/21
2001	이소희	여	회계	0202	1993/12/13
2002	이소희	여	회계	0202	1996/04/30
2003	이소희	여	회계	0202	1992/03/03
2007	이소희	여	회계	0001	1992/09/05
2008	이소희	여	회계	0001	1992/03/03
3002	이소희	여	회계	1002	2001/01/29
0309	이소희	여	회계	0302	2011/01/03
3001	이소희	여	회계	1001	1998/10/17
0702	이소희	여	회계	0301	2017/01/09
0269	이소희	여	회계	0301	2015/05/21
0401	이소희	여	회계	1001	2008/03/13
0801	이소희	여	회계	1001	2000/10/09
0120	이소희	여	회계	1002	1999/09/05

21 행이 선택되었습니다.

SQL>

MGR

HDATE

SAL

COMM

DNO

33001	김선유	3200	300	남	분석
31001	98/10/17			10	
40702	김민지			남	회계
40301	17/01/09	1100	60	02	
30269	권나현			여	분석
10301	15/05/21	2600	1900	10	
2ENO	ENAME			SEX	JOB
2MGR	HDATE	SAL	COMM	DNO	
310401	김진성			남	회계
21001	08/03/13	3200	1000	10	
320801	천유정			여	분석
41001	00/10/09	2900	600	02	
0120	김경현			남	지원
1002	99/09/05	4000	2500	20	

21 행이 선택되었습니다.

SQL>

sqlplus 출력제어

- SET 명령

SET LIN[ESIZE] ##

- 한 라인에 출력할 문자의 수

SET PAGES[IZE] ##

- 한 페이지에 출력할 라인의 수
 - . ROW의 개수가 아님
- 페이지마다 헤더가 출력된다.

- 컬럼 형식 지정

COL[UMN] 컬럼 FORMAT [A## | 숫자형식]

- 컬럼의 출력 형식을 지정한다.
- 문자 컬럼 : A##로 출력 너비를 지정한다.
- 숫자 컬럼 : 0,9를 이용해서 출력 형식을 지정한다.
 - . 단일행 함수에서 자세히 다룬다.

COL[UMN] 컬럼 CLEAR

- 컬럼의 출력 형식을 삭제한다.

설정 방법

- 화면 제어 설정 방법
 - ORACLE_HOME/sqlplus/admin/glogin.sql
 - . sqlplus가 실행될때 자동으로 실행되는 스크립트 파일
 - sqlplus 실행 디렉토리에 스크립트 파일을 만들고 직접 실행한다.
 - . login.sql : school.sql 파일과 같이 배포함
- login.sql 파일의 내용

```
-- ALTER SESSION SET NLS_DATE_FORMAT='YYYY/MM/DD';  
SET LINE 80  
SET PAGES 200  
COL ACCOUNT_STATUS FORMAT A20  
COL CNAME FORMAT A16  
COL CNO FORMAT A10  
COL COLUMN_EXPRESSION FORMAT A60  
COL COLUMN_NAME FORMAT A15  
COL COMMENTS FORMAT A65  
COL CONSTRAINT_NAME FORMAT A20  
.....
```

연결 연산자와 중복의 제거

학습목표

1. 연결 연산자를 이용해서 검색한다.
2. 연산자 우선순위를 이해한다.
3. 중복 데이터를 검색한다.
4. 중복 회피방법에 대해서 이해한다.

연결 연산자를 이용한 검색

```
SELECT [컬럼 | '리터럴'] ||, .....  
FROM 테이블;
```

- 연결 연산자(||)는 리터럴이나 컬럼을 하나의 문자열로 검색한다.
- 리터럴이란 SQL문에 쓰인 문자, 숫자, 날짜를 의미한다.
 - . 리터럴은 숫자를 제외하고 반드시 단일 인용부호를 사용한다.
- 연결 연산자는 SELECT 문을 이용 다양한 문장을 만들 때 많이 사용된다.
- 리터럴을 제외한 것들은 예약어이다.
 - . SELECT, FROM, WHERE, ORDER,

연결 연산자를 이용한 검색

```
SQL> SELECT ename || sa1 이름_급여  
2 FROM emp;
```

```
SQL> SELECT ename || ' ' || sa1 이름_급여  
2 FROM emp;
```

```
SQL> SELECT ename || '의 업무는 ' || job || '입니다.'  
2 FROM emp;
```

주의할 연결 연산자

```
SQL> SELECT ename || ' ' || sal+100 이름_급여  
2 FROM emp;
```

```
SQL> SELECT ename || ' ' || sal*1.1 이름_급여  
2 FROM emp;
```

- 에러 : ORA-01722: 수치가 부적합합니다.
- 연산자 우선순위

- 연산자 우선순위

$*, / > || > +, - > \dots$

- 연산자 우선순위와 무관하게 작성!!!

주의할 연결 연산자

Ex. '## ##님의 년봉은 ###입니다.' 형으로 사원의 급여를 검색한다.

```
SQL> SELECT eno||ename ||'님의 년봉은 '  
2          ||(sal*12+nvl(comm,0))||'입니다.'  
3 FROM emp;
```

- 연산자가 중첩되는 경우 반드시 괄호를 이용해서 가독성을 확보한다.
- 연산자 우선순위를 고려하지 않아도 가독성이 확보되어야 한다.

중복제거

```
SELECT [DISTINCT | ALL] .....  
FROM 테이블;
```

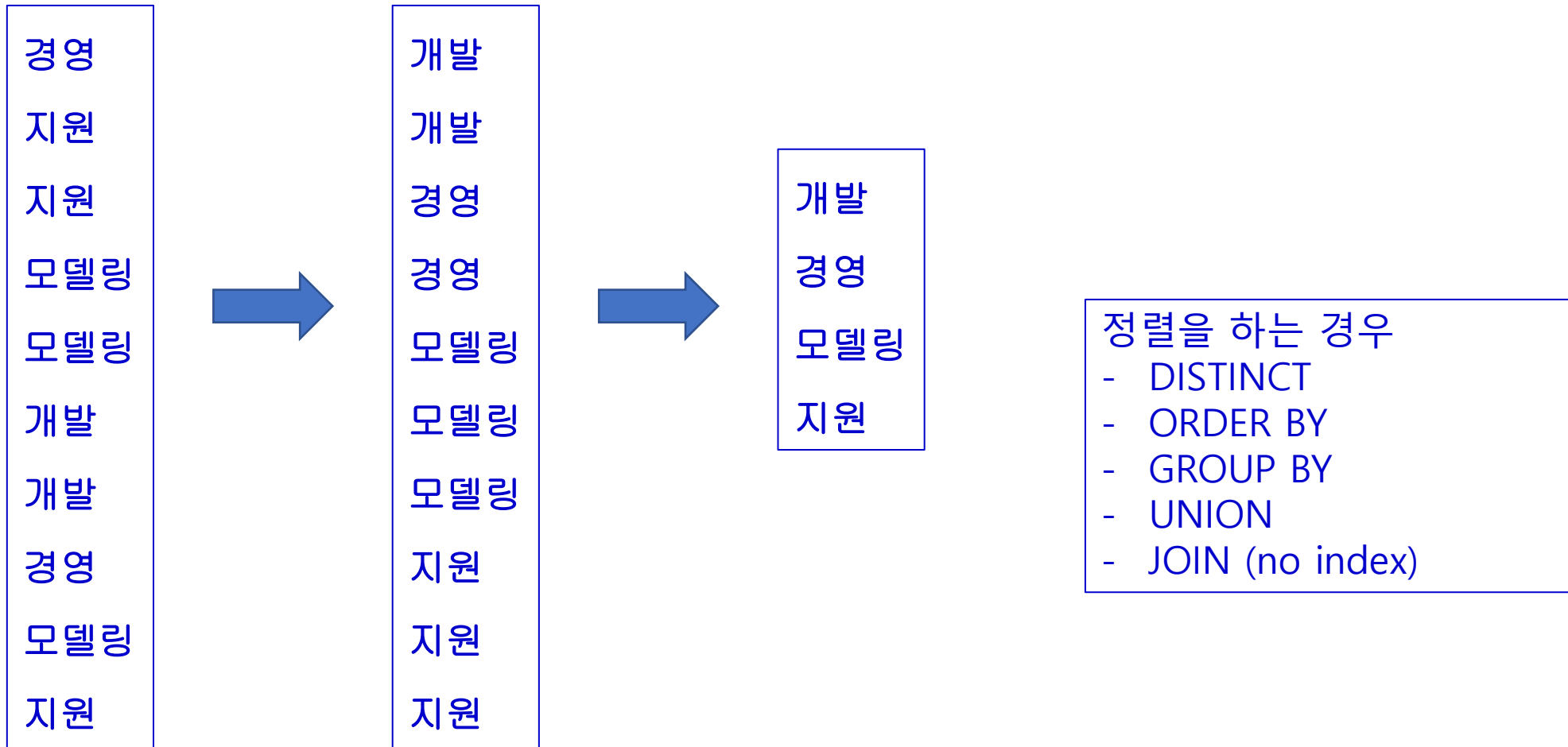
- DISTINCT : 중복된 행은 한번만 검색한다.
 . 출력 행을 기준으로 한다.
- ALL : 중복된 모든 행을 검색한다. (default)

```
SQL> SELECT job 업무  
2 FROM emp;
```

```
SQL> SELECT DISTINCT job 업무  
2 FROM emp;
```

중복과 정렬

- 다음 값에서 중복된 값을 정리한다.



실습

1. '____학과인 ____학생의 현재 평점은 __입니다.' 형태로 학생의 정보를 출력한다.
2. '____과목은 ____학점 과목입니다.' 형태로 과목의 정보를 출력한다.
3. '____교수는 ____학과 소속입니다.' 형태로 교수의 정보를 출력한다.
4. 학교에는 어떤 학과가 있는지 검색한다.(학생 테이블 기반으로 검색한다.)
5. 학교에는 어떤 학과가 있는지 검색한다.(교수 테이블 기반으로 검색한다.)
6. 교수의 지위는 어떤 것들이 있는지 검색한다.
7. '____학생의 4.5 환산 평점은 ____입니다.' 형태로 학생의 환산 평점을 출력한다.
8. '____과목의 담당 교수 번호는 ____입니다.' 형태로 과목의 정보를 출력한다.
9. 학교에 개설된 과목들은 몇 학점인지 검색한다.
10. 학생들이 수강중인 과목의 과목번호를 검색한다.

정렬을 이용한 자료 검색

학습목표

1. ORDER BY절을 이용해서 검색 행을 정렬한다.
2. 다양한 방법으로 정렬을 수행한다.
3. 정렬을 이용해서 묶음 검색한다.

정렬 검색

```
SELECT * | [DISTINCT] 컬럼|수식 [AS 별명], .....  
FROM 테이블  
ORDER BY 컬럼 [ASC|DESC], 컬럼 [ASC|DESC], .....
```

- ORDER BY : 지정된 컬럼을 대상으로 정렬을 수행한다.
 - ASC는 오름차순 정렬, DESC는 내림차순 정렬을 지정한다.
 - . default : ASC
 - 정렬대상인 컬럼이 여러 개인 경우 앞에 지정한 컬럼이 동일 한 값에 대해서 뒤에 지정된 컬럼으로 정렬된다
 - 정렬하지 않는 SELECT문의 출력 순서는 의미가 없다.
 - 정렬 대상으로는 컬럼 이외에 다음의 것들이 사용 가능하다.
 - . 별명, 수식, 검색 항목 순서
- 원칙 : RDB에서 행의 순서나 컬럼의 순서는 의미가 없다.
 - 순서는 Application이다.

정렬 검색

```
SQL> SELECT eno 사번, ename 이름, sal 급여  
2 FROM emp  
3 ORDER BY sal DESC;
```

```
SQL> SELECT eno 사번, ename 이름, sal 급여  
2 FROM emp  
3 ORDER BY 급여 DESC;
```

```
SQL> SELECT eno 사번, ename 이름, sal 급여  
2 FROM emp  
3 ORDER BY 3 DESC;      ← 가급적 쓰지말자!!
```

정렬 검색

```
SQL> SELECT eno 사번, ename 이름  
2  FROM emp  
3  ORDER BY sal*12+nvl(comm,0) DESC;
```

- SQL문은 의미가 있으나 검색 결과는 의미를 알기 어렵다.
- $sal*12+nvl(comm,0)$ 를 검색하거나 적당한 별명을 추가한다.

```
SQL> SELECT eno 사번, ename 이름, sal 급여, comm 보너스  
2  FROM emp  
3  ORDER BY sal DESC, comm DESC;
```

- 여러 컬럼으로 정렬하는 것은 순서가 중요하다.

묶음 검색

- 업무별로 사원의 급여를 검색한다.

```
SQL> SELECT job 업무, eno 사번, ename 이름, sal 급여  
2   FROM emp  
3   ORDER BY 업무;
```

- 부서별로 사원의 급여를 검색한다.

```
SQL> SELECT dno 부서번호, eno 사번, ename 이름, sal 급여  
2   FROM emp  
3   ORDER BY 부서번호, sal DESC;
```

의미 없는 검색

```
SQL> SELECT dno 부서번호, eno 사번, ename 이름, sal 급여  
2   FROM emp  
3   ORDER BY sal, dno;
```

- 결과에 ORDER BY절의 의미가 없다.

- 카디널리티(Cardinality)
 - 중복 정도
 - 성별 : 카디널리티가 낮다.
 - 이름 : 카디널리티가 높다.

실습

1. 성적순으로 학생의 이름을 검색한다.
2. 학과별 성적순으로 학생의 정보를 검색한다.
3. 학년별 성적순으로 학생의 정보를 검색한다.
4. 학과별 학년별로 학생의 정보를 성적순으로 검색한다.
5. 학점 순으로 과목 이름을 검색한다.
6. 각 학과별로 교수의 정보를 검색한다.
7. 지위별로 교수의 정보를 검색한다.
8. 각 학과별로 교수의 정보를 부임일자 순으로 검색한다.
9. 급여가 10% 인상된 경우 부서별로 각 사원의 연봉을 연봉순으로 검색한다.
10. 보너스가 100% 인상된 경우 업무별로 각 사원의 연봉을 연봉순으로 검색한다.

WHERE절을 이용한 조건 검색

학습목표

1. WHERE절에 대해서 학습한다.
2. 조건문에 대해서 학습한다.
3. 날짜 형식에 대해서 학습한다.
4. 틀리기 쉬운 조건 기술에 대해서 이해한다.
5. 관계 연산자를 이용하는 다양한 조건을 학습한다.

SELECT문의 기능

프로젝션(Projection)

선택(Selection)

조인(Join)

조건 검색

```
SELECT * | 컬럼, .....  
FROM 테이블  
WHERE 검색조건  
ORDER BY 컬럼 [ASC|DESC], 컬럼 [ASC|DESC], .....
```

- 조건을 통해 테이블의 일부 행만을 검색한다.
- 조건은 컬럼을 대상으로 데이터를 제한한다.
- 조건에는 비교 연산자가 사용된다.
 - . =, <, >, <=, >=
 - . !=, <>, ^=
- 지문에 대한 이해가 필요하다.
 - . 테이블의 구성을 기억해야 한다.

다양한 조건 검색

- 직원중에 급여가 4000 이상인 직원을 검색한다.
- 이름이 문시현인 직원을 검색한다.
- 10번 이외 부서 정보를 검색한다.

```
SQL> SELECT eno 사번, ename 이름, sal 급여  
2 FROM emp  
3 WHERE sal >= 4000;
```

```
SQL> SELECT * FROM emp  
2 WHERE ename = '문시현';
```

```
SQL> SELECT * FROM dept  
2 WHERE dno != '10';
```


잘못된 조건 검색

```
SQL> SELECT * FROM emp  
2  WHERE dno != 10;
```

- dno에 대한 조건 기술이 틀림
- 결과는 정상적으로 검색됨
- 오라클(대부분의 RDBMS)는 자동 형변환 기능을 수행함
- 절대 방치해서는 안됨!!!

다양한 조건 검색

- 급여가 4000 이상인 사원을 급여 순으로 검색한다.
- 년봉이 30000 이상인 사원을 부서별로 검색한다.

```
SQL> SELECT eno 사번, ename 이름, sal 급여  
2 FROM emp  
3 WHERE sal >= 4000  
4 ORDER BY sal DESC;
```

```
SQL> SELECT dno 부서, eno 사번,  
2          ename 이름, sal*12+nvl(comm,0) 년봉  
3 FROM emp  
4 WHERE sal*12+nvl(comm,0) >= 30000 ← 년봉 >= 30000 사용 불가!!  
5 ORDER BY dno;
```

날짜 검색

- 입사일이 1996년 이후인 사원의 정보를 검색한다.

```
SQL> SELECT * FROM emp  
2  WHERE hdate >= '1996/01/01';
```

- 날짜 형식 변경과 확인

```
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY/MM/DD';
```

```
SELECT *  
FROM NLS_SESSION_PARAMETERS  
WHERE PARAMETER = 'NLS_DATE_FORMAT';
```

- YYYY/MM/DD:HH24:MI:SS
- DD-Mon-RR

관계 연산자와 부정 연산자

- 조건이 여러 개인 경우나 부정 검색인 경우 관계 연산자가 사용된다.
 - AND, OR, NOT
- 회계 업무를 담당하는 여성 사원을 검색한다.
- 회계 업무와 분석 업무 담당 사원을 검색한다.
- 회계 업무 이외 업무는 어떤 업무가 있는지 검색한다.

```
SQL> SELECT * FROM emp  
2 WHERE job='회계' AND sex='여';
```

```
SQL> SELECT * FROM emp  
2 WHERE job='회계' OR job='분석';
```

```
SQL> SELECT DISTINCT job FROM emp  
2 WHERE NOT job='회계';      ← job != '회계'
```

실습

1. 화학과 학생을 검색한다.
2. 평점이 2.0 미만인 학생을 검색한다.
3. 권현 학생의 평점을 검색한다.
4. 정교수의 명단을 검색한다.
5. 화학과 소속 교수의 명단을 검색한다.
6. 송강 교수의 정보를 검색한다.
7. 학년별로 화학과 학생의 성적을 검색한다.
8. 2000년 이전에 부임한 교수의 정보를 부임일순으로 검색한다.

다양한 조건 검색

NULL, BETWEEN, IN, LIKE

학습목표

1. NULL 비교 연산자를 학습한다.
2. BETWEEN ... AND 연산자를 학습한다.
3. IN 연산자를 학습한다.
4. LIKE 연산자를 학습한다.

NULL과 비교 연산

- NULL은 비교 연산자(=, <, >)를 사용할 수 없다.
- NULL에 대한 직접 검색은 다음 연산자를 이용한다.
 - 컬럼 IS NULL
 - 컬럼 IS NOT NULL
- 보너스가 결정되지 않은 사원을 검색한다.

```
SQL> SELECT * FROM emp  
2  WHERE comm IS NULL;
```


BETWEEN..AND 연산자

```
SELECT * | 컬럼, .....  
FROM 테이블;  
WHERE 컬럼 BETWEEN 값1 AND 값2  
ORDER BY 컬럼 .....
```

- '컬럼 >= 값1 AND 컬럼 <= 값2' 와 동일하다.
- 컬럼의 값이 값1에서 값2 사이의 값을 검색한다.
- 값1은 반드시 값2보다 작아야 한다.
- AND 연산자로 환원 가능하지만 가독성이 떨어진다.

BETWEEN..AND 연산자

- 급여가 1000에서 2000 이내인 사원을 검색한다.
- 1992년에서 1996년 사이에 입사한 사원을 검색한다.

```
SQL> SELECT * FROM emp  
2  WHERE sal BETWEEN 1000 AND 2000;
```

```
SQL> SELECT * FROM emp  
2  WHERE hdate BETWEEN '1992/01/01' AND '1996/12/31'  
3  ORDER BY hdate;
```

- sal >= 1000 AND sal <= 2000
- hdate >= '1992/01/01' AND hdate <= '1996/12/31'

잘못 기술된 BETWEEN

```
SQL> SELECT * FROM emp  
2  WHERE sal BETWEEN 2000 AND 1000;
```

- BETWEEN 연산자 이용시 반드시 작은 값이 먼저 와야 한다.
- 에러가 발생하지 않고 '선택된 레코드가 없습니다.'가 출력된다.
- 'BETWEEN -3 AND -4'와 같은 실수가 가능하다.

IN 연산자

```
SELECT * | 컬럼, .....  
FROM 테이블;  
WHERE 컬럼 IN (값1, 값2 ...)  
ORDER BY 컬럼 .....
```

- 값 중에 일치하는 값이 하나라도 있으면 검색된다.
- OR 연산자와 = 연산자로 교환 가능하다.
- 가독성이 매우 우수하다.
- IN 연산자는 나중에 서브 쿼리, 파티션 등의 여러 기능에 사용되는 중요한 연산자다.

IN 연산자

- 개발이나 분석 업무를 담당하는 사원을 검색한다.
- 10, 20번 부서 사원을 검색한다.

```
SQL> SELECT * FROM emp  
2  WHERE job IN ('개발', '분석')  
3  ORDER BY JOB;
```

```
SQL> SELECT * FROM emp  
2  WHERE dno IN ('10', '20')  
3  ORDER BY dno;
```

- job='개발' OR job='관리'
- dno='10' OR dno='20'

LIKE 연산자

```
SELECT * | 컬럼, .....  
FROM 테이블;  
WHERE 컬럼 LIKE '리터럴'  
ORDER BY 컬럼 .....
```

- LIKE 연산자는 패턴 연산자의 일종으로 일부 문자열만을 이용해서 대상을 검색한다.
- 와일드 문자를 사용한다.
 - . % : 문자열 공백을 포함한다.
 - . _ : 문자

LIKE 연산자

- 와일드 문자

예	설명	검색 대상
'경%'	'경'으로 시작하는 모든 문자열	경, 경제, 경범죄, 경영학과 ...
'%과'	'과'로 끝나는 모든 문자열	과, 다과, 화학과, 물리학과 ...
'%김%'	'김'이란 글자가 들어간 모든 문자열	김, 김씨, 돌김, 되새김질 ...
'화_'	'화'로 시작하는 두글자 단어	화학, 화약, 화상 ...
'__화'	'화'로 끝나는 세 글자 단어	무궁화, 해당화, 운동화 ...
'_동_'	'동'이 가운데 들어간 세글자 단어	원동기, 전동차 ...
'%이#%%'	ESCAPE '#' 으로 지정한 경우, '이%' 가 포함된 문자열	
'_#%_'	ESCAPE '#' 으로 지정한 경우, '%'가 포함된 세 글자 단어	

LIKE 연산자

- 김씨 성을 가진 사원
- 이름이 '하늘'인 사원
- 성과 이름이 각각 한 글자인 사원을 검색한다.

```
SQL> SELECT * FROM emp  
2  WHERE ename LIKE '김%';
```

```
SQL> SELECT * FROM emp  
2  WHERE ename LIKE '%하늘';
```

```
SQL> SELECT * FROM emp  
2  WHERE ename LIKE '___';
```


LIKE 연산자

- 이름에 '%' 문자가 포함된 사원을 검색한다.

```
SQL> SELECT eno, ename  
2   FROM emp  
3   WHERE ename LIKE '%#%' ESCAPE '#';
```

실습

1. 1999년에서 2001년까지 부임한 교수의 명단을 검색한다.
2. 정교수와 조교수를 검색한다.
3. 학점수가 1학점, 2학점인 과목을 검색한다.
4. 1,2 학년 학생 중에 평점이 2.0에서 3.0사이인 학생을 검색한다.
5. 물리, 화학과 학생 중 4.5 환산 평점이 3.5에서 4.0 사이인 학생을 검색한다.
6. 물리, 화학과 학생의 정보를 학년별 성적순으로 검색한다.
7. 물리, 화학과 교수 중에 1999년에서 2000년 사이에 부임한 교수의 정보를
직위별로 검색한다.
8. 교수가 지정되지 않은 과목 중에 학점이 3학점인 과목을 검색한다.
9. 화학 관련된 과목 중 학점이 2학점 이하인 과목을 검색한다.
: 화학 관련 과목은 과목 명에 화학이 들어간 과목을 의미한다.
10. 화학과 학생 중에 성이 권씨인 학생을 검색한다.